

PROJECT 4 REPORT

Junyao Han, Junrui Ding

Overall Project Description

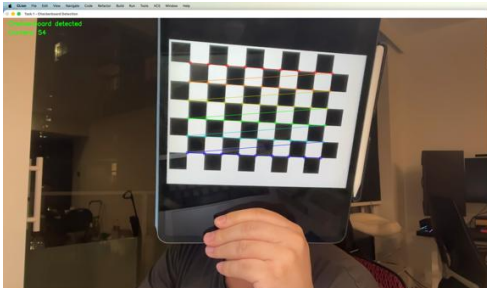
This project focuses on camera calibration and pose estimation for augmented reality. The system first detects a checkerboard target in a live video stream, extracts the corner locations, and saves selected frames for calibration. Using the corresponding 2D image points and 3D world points, the camera is calibrated to estimate its intrinsic parameters and distortion coefficients. After calibration, the intrinsic parameters are loaded from a file and used together with solvePnP to estimate the pose of the checkerboard in real time. The first part of the project establishes the calibration pipeline, and the later part uses that calibration to recover the board position and orientation relative to the camera.

Task 1: Detect and Extract Target Corners

For this project, I used a checkerboard target. The checkerboard has 9 columns and 6 rows of internal corners, so the system should detect 54 corners when the pattern is fully visible.

I used OpenCV's findChessboardCorners to detect the checkerboard corners, cornerSubPix to refine the corner locations, and drawChessboardCorners to display them in the video stream. The detected corners were drawn on the image in real time, and the program also displayed how many corners were found. In successful detections, the program consistently found 54 corners, which matched the expected pattern size.

The system worked well when the checkerboard was clearly visible and the lighting was reasonably even. Detection became less stable when there was glare on the screen, motion blur, or a more extreme viewing angle. Since I displayed the checkerboard on a tablet, reflections on the screen were one of the main practical challenges.

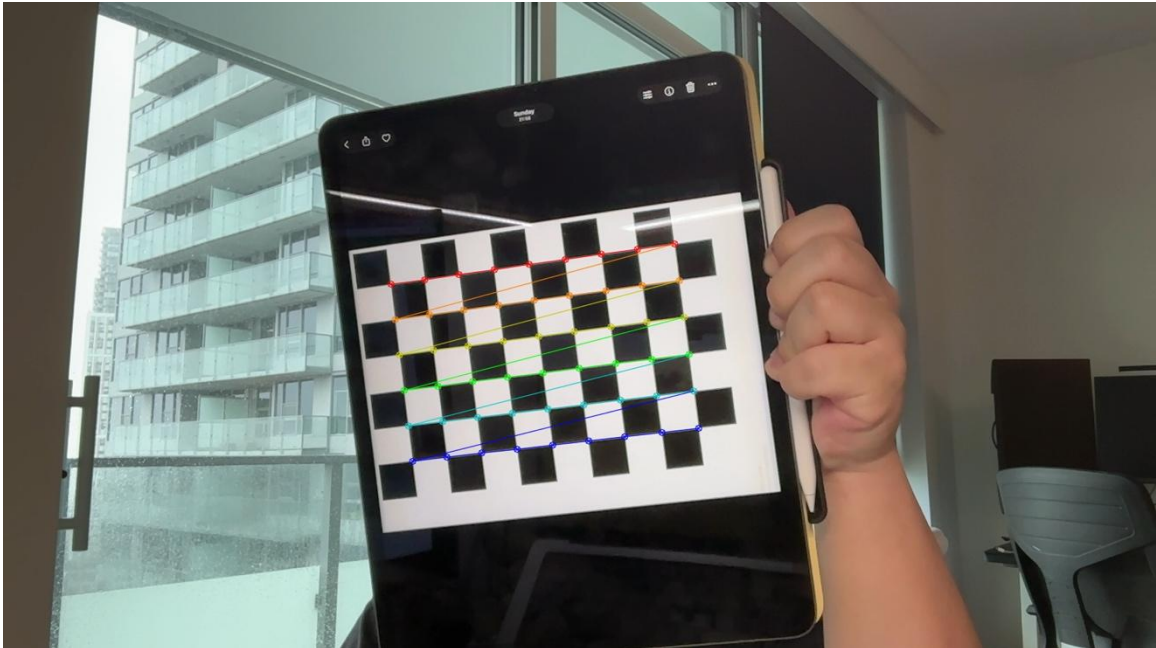


Task 2: Select Calibration Images

The program was extended so that the user could press `s` to save a calibration sample. Each saved sample stored two things: the detected 2D image corners and the corresponding 3D world points of the checkerboard.

The 3D world points were defined in checkerboard square units. I used the upper-left checkerboard corner as the origin $(0, 0, 0)$, then assigned points across the board as $(1, 0, 0)$, $(2, 0, 0)$, and so on. For the next row, the coordinates started at $(0, -1, 0)$, which matched the coordinate system described in the assignment. This produced one `point_set` for each saved checkerboard image, and the number of 3D points matched the number of detected 2D corners in every saved sample.

The program also saved the calibration images themselves so they could be included in the report. To make the calibration more reliable, I selected frames from different distances and viewing angles instead of saving many nearly identical images.



Task 3: Calibrate the Camera

After saving enough calibration frames, I used OpenCV's `calibrateCamera` function to estimate the camera intrinsics and distortion coefficients. The camera matrix was initialized as a 3×3 matrix of type `CV_64FC1`, following the assignment instructions. The program printed the camera matrix and distortion coefficients before and after calibration, along with the final

reprojection error.

The final camera matrix was:

```
[ 1417.3216471764733, 0., 953.24709193285139  
  0., 1417.3216471764733, 546.373500948794  
  0., 0., 1. ]
```

The final distortion coefficients were:

```
[ -0.028011520407719665, 0.25932784651920293,  
  0.00022216517009497654, 0.0012898248657683274,  
  -0.59795756574535142 ]
```

The final reprojection error was: 0.22702857323754971

This is a good result because the reprojection error is well below 1 pixel. The estimated focal lengths in the x and y directions are equal, and the principal point is close to the image center, so the calibration result looks reasonable. The intrinsic parameters were then written to a file for later use in pose estimation.

```
1  %YAML:1.0  
2  ---  
3  camera_matrix: !!opencv-matrix  
4      rows: 3  
5      cols: 3  
6      dt: d  
7      data: [ 1417.3216471764733, 0., 953.24709193285139, 0.,  
8              1417.3216471764733, 546.373500948794, 0., 0., 1. ]  
9  distortion_coefficients: !!opencv-matrix  
10     rows: 1  
11     cols: 5  
12     dt: d  
13     data: [ -0.028011520407719665, 0.25932784651920293,  
14             0.00022216517009497654, 0.0012898248657683274,  
15             -0.59795756574535142 ]  
16  reprojection_error: 0.22702857323754971
```

Task 4: Calculate Current Position of the Camera

For the next stage, I wrote a separate program that reads the saved intrinsic calibration file and starts a live video loop. In each frame, the program detects the checkerboard target, reconstructs the corresponding 3D world points, and uses OpenCV's solvePnP to estimate the board pose relative to the camera.

The program printed both the rotation vector (rvec) and translation vector (tvec) in real time. One example result is shown below:

rvec:

```
[3.104625877179128;  
0.02089780716290655;  
-0.05018304855010831]
```

tvec:

```
[-1.970993766403218;  
-2.402385542621446;  
22.68593269261844]
```

As I moved the checkerboard side to side, the translation vector changed accordingly, especially in the x component. When the checkerboard moved farther away from the camera, the z value increased. When I tilted the board, the rotation vector also changed. These changes matched the actual motion of the checkerboard, so the pose estimates made sense and appeared stable during testing.

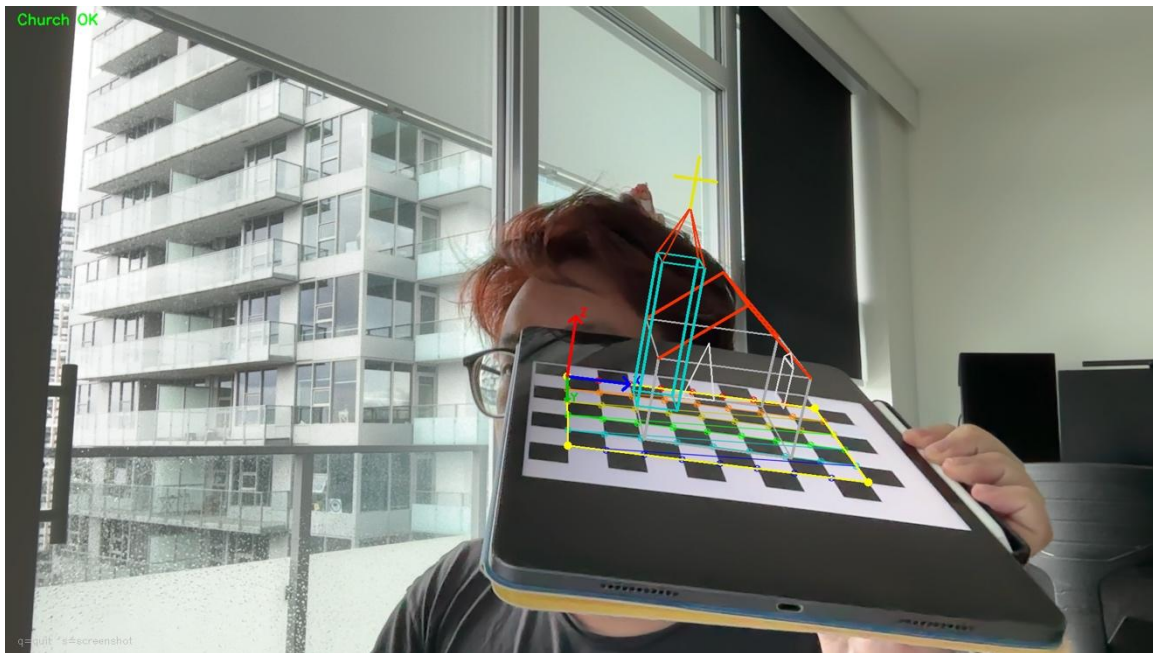
Task 5: Project Outside Corners and 3D Axes

For Task 5, I used OpenCV's projectPoints function to project 3D world-space points onto the image plane in real time. This task served as a verification step: if the projected points align correctly with the physical checkerboard in the image, it confirms that the calibration parameters and pose estimate from Task 4 are both accurate.

I projected two sets of points. The first set consisted of the four outer corners of the checkerboard target. The 9×6 internal corner grid spans $X = [0, 8]$ and $Y = [0, -5]$ in world coordinates, so the four outer corners are $(0, 0, 0)$, $(8, 0, 0)$, $(8, -5, 0)$, and $(0, -5, 0)$. These were projected and connected with yellow lines to form a rectangle, with filled circles drawn at each corner.

The second set consisted of the three coordinate axis endpoints: $(2, 0, 0)$ for the X axis, $(0, -2, 0)$ for the Y axis, and $(0, 0, 2)$ for the Z axis, all relative to the origin at $(0, 0, 0)$. These were drawn as colored arrows originating from the board origin: blue for X, green for Y, and red for Z. The Z axis is especially useful for verification because it should visibly point out of the board plane toward the camera.

As shown in the figure below, the yellow rectangle tightly bounds the checkerboard, and the projected XYZ axes are aligned correctly with the board origin and orientation. As the camera or board moved, both the rectangle and the axes tracked correctly in real time, confirming that pose estimation was working as expected before proceeding to Task 6.



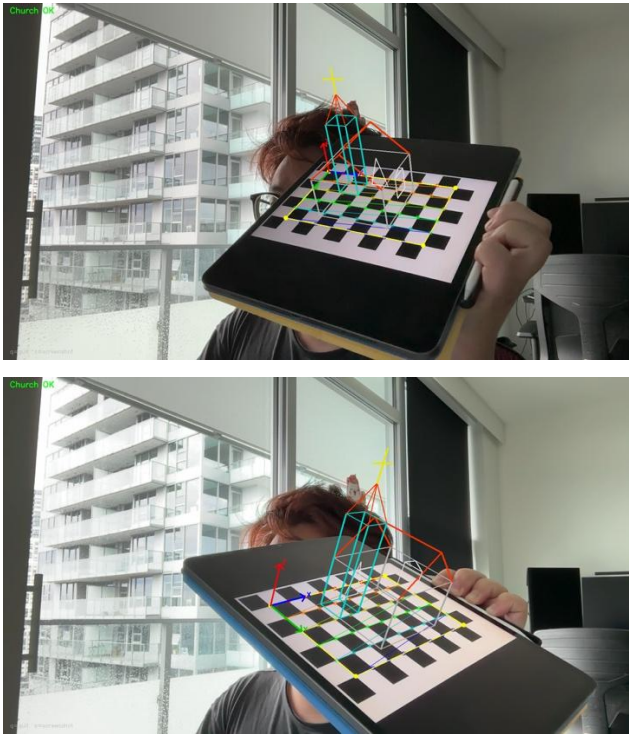
Task 6: Virtual Object — 3D Church

For the virtual object, I designed a 3D church in world space using 29 vertices and approximately 35 line segments, then projected all vertices to image space using `projectPoints` and connected them with `cv::line`. The object uses four colors to distinguish structural components.

The church consists of a main nave with a gable roof, a bell tower on the front-left corner, a cross

mounted on the tower spire, a pointed door arch on the front face, and a side window with an arched top on the right wall. In world coordinates, the board spans $X = [0, 8]$ and $Y = [0, -5]$, and the church occupies $X = [2, 6]$, $Y = [-1, -4.5]$, floating slightly above the board surface at $Z = 0.2$. The gable ridge peaks at $Z = 4.2$, the bell tower top ring sits at $Z = 4.5$, the spire peak reaches $Z = 6.0$, and the cross top extends to $Z = 7.4$. The gray walls and floor form the nave structure, cyan lines mark the bell tower, red lines form the roof and spire edges, and yellow marks the cross. White lines draw the door arch and side window.

Because every vertex is defined in world space and projected fresh each frame using the current `rvec` and `tvec` from `solvePnP`, the church automatically maintains correct position and orientation as the camera or board moves. The structure stays anchored to the checkerboard from any viewing angle.



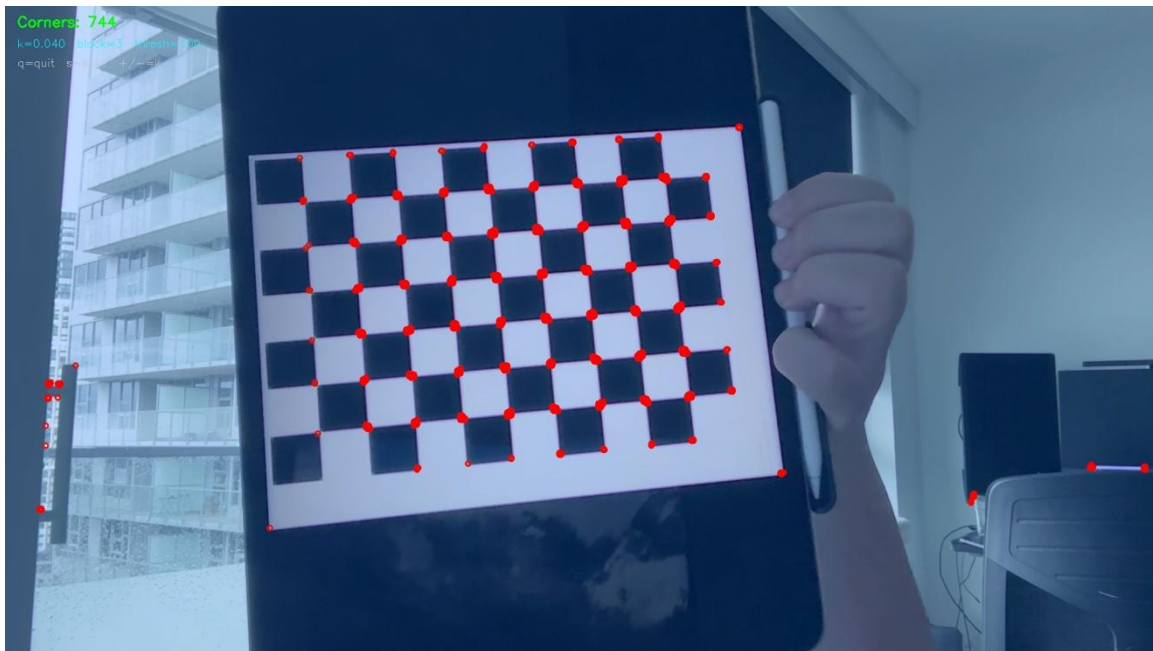
Task 7: Detect Robust Features

For Task 7, I wrote a separate standalone program (`project4_features`) that detects Harris corners on a live camera feed without requiring a checkerboard or calibration. The program runs `cv::cornerHarris` on every frame and overlays the results in two ways: a JET colormap heat map blended into the background to show the distribution of response strength across the entire image, and red circles drawn at every pixel whose normalized response exceeds the threshold. To keep

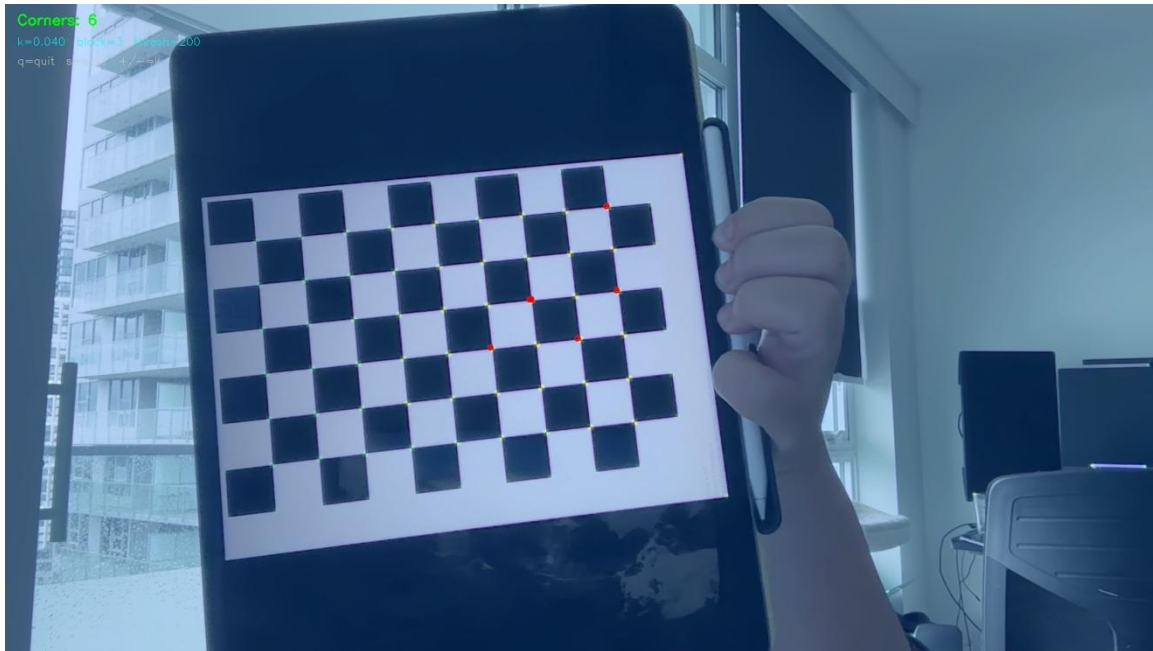
the interface responsive, detection runs on a frame scaled to 40% of the original resolution, and corner coordinates are scaled back up before drawing.

Three parameters can be tuned interactively. A threshold tracker (0–255) controls how strong a response must be before it is marked as a corner. A block size tracker sets the neighbourhood size used by the Harris detector (mapped to odd values 1, 3, 5, 7, 9): larger neighbourhoods average gradients over a wider area, smoothing out noise so that only genuinely structured corners survive. The Harris sensitivity parameter k can be adjusted with the + and – keys: lower k makes the detector more sensitive, while higher k suppresses weaker responses.

The images below compare threshold 100 and threshold 200 with all other parameters held constant. At threshold 100, many corners are detected across the scene. Notably, corners appear not only at the checkerboard's internal intersections but also on other objects in the frame, such as the edges of the monitor visible in the background, demonstrating that Harris responds to any region where intensity changes sharply in two directions, regardless of what the object is.



[task7_harris_100.png — threshold=100, many corners detected]



[task7_harris_200.png — threshold=200, only strong corners remain]

At threshold 200, only the strongest corners remain. Interestingly, even though all checkerboard internal corners should theoretically have equal Harris response — they are all black-white intersections of the same geometry — the surviving corners are not evenly distributed across the board. This is because real-world conditions break the theoretical symmetry: uneven lighting makes corners near the window stronger than those in shadow, perspective distortion compresses the far corners so their gradients are weaker in image space, and screen glare reduces local contrast in affected regions. This shows that Harris response strength reflects image conditions as much as geometric structure, which is an important practical limitation.

Harris corners are one possible foundation for marker-free augmented reality. The key distinction from the checkerboard pipeline used in Tasks 1–6 is not whether corners can be detected — Harris can find the same 54 internal corners on a checkerboard that `findChessboardCorners` finds — but whether those detections can be used for pose estimation. `findChessboardCorners` exploits the board's known structure to return corners in a fixed order, so the k -th detected point always corresponds to the k -th known 3D world coordinate, enabling `solvePnP` directly. Harris returns an unordered set of points with no 3D information attached. To use them for AR, two additional steps would be needed: a feature descriptor such as SIFT or ORB to establish which point in one frame matches which point in the next, and a way to associate matched points with known 3D positions, for example through a prior reconstruction or depth sensor. Harris detection is therefore

the first step in a marker-free pipeline, but matching and 3D structure recovery are required before solvePnP can be called.

Extensions

Extension 1: Two Targets with a Dynamic Arrow Sign

The two-target extension places two separate AR objects in the scene simultaneously. Board A uses the standard 9×6 checkerboard and displays the church from Task 6. Board B uses a 5×4 checkerboard displayed on a second screen, and displays a road-sign-style arrow that dynamically rotates to point toward the church on Board A.

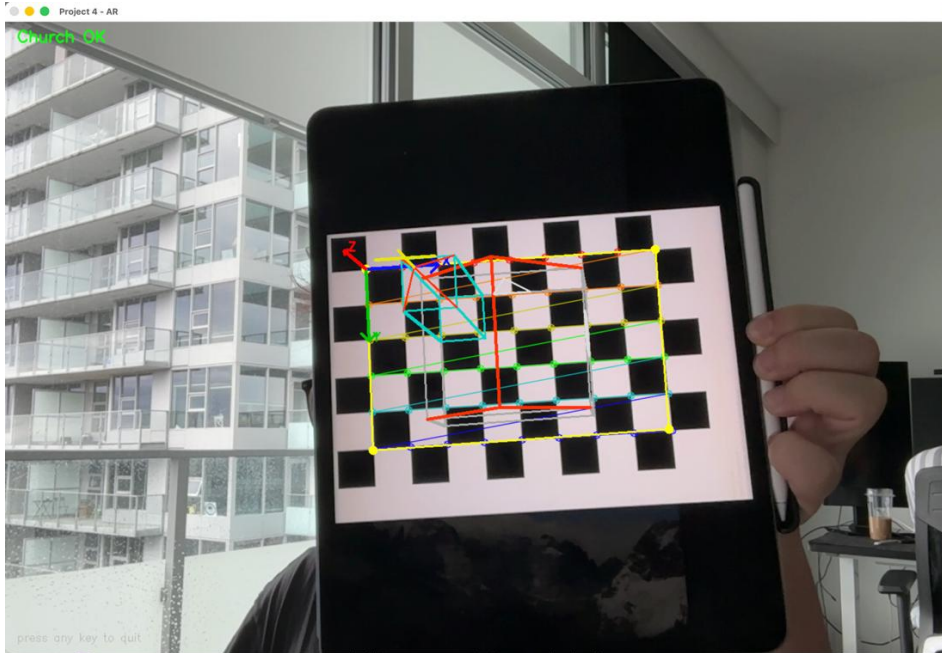
The direction computation works as follows. Each frame, solvePnP is called independently for Board A and Board B, producing separate rvec and tvec for each. The translation vector tvec encodes the position of each board's origin in camera space. The vector from the sign to the church is computed as $\text{dir_cam} = \text{tvec_church} - \text{tvec_sign}$, giving a direction in camera coordinates. This is then transformed into Board B's local coordinate frame by multiplying with R_{sign}^T , the transpose of Board B's rotation matrix. The in-plane angle is extracted with $\text{atan2}(\text{local_y}, \text{local_x})$ and used to rotate the arrow vertices around the Z axis before projecting them. As either board is moved, the arrow recomputes its direction every frame and rotates accordingly, always pointing toward the church.

This extension required detecting two different pattern sizes in each frame. To avoid the drawings from Board A interfering with Board B's detection, both boards are detected on a clean copy of the frame captured before any AR drawing takes place. The same clean frame is also used for the performance optimization described below.



Extension 2: Static Image Mode

The program also supports processing a pre-captured image instead of a live camera feed. When a file path is provided as a command-line argument, the program reads the image, runs checkerboard detection and pose estimation once, draws the church and coordinate axes, and displays the result until a key is pressed.



This mode demonstrates that the AR pipeline is not limited to real-time video. Any image captured with the same camera whose intrinsics are stored in `intrinsics.yml` can have virtual objects inserted into it after the fact.

Reflection

Junyao: This project helped me understand how camera calibration and pose estimation connect together in a full pipeline. Before this project, I mostly saw calibration as a formula-based topic, but implementing it made it much clearer. I also learned that small practical issues, such as glare, blur, and viewpoint diversity in the calibration images, can strongly affect the final result. The most useful takeaway for me was seeing how 2D image points and 3D world points are linked through calibration and then used again in `solvePnP` to recover pose.

Junrui: Working on Tasks 5–7 and the extensions gave me a much clearer understanding of the full AR pipeline, and I genuinely enjoyed the creative freedom in this part of the project. For the extensions, I wanted two targets in the scene: a church on one board and an arrow sign on the other that always points toward the church. Getting the direction math right took some time — I had to work through how to compute the vector from one board's `tvec` to the other and then rotate it into the sign board's local coordinate frame before extracting the angle.

Some of the most interesting moments came from unexpected problems. To keep the church anchored to one specific board rather than jumping between the two, the simplest fix turned out to be using a different pattern size for each board, so `findChessboardCorners` could never confuse them. Testing the static image mode also caught me off guard: I photographed my phone screen with the MacBook camera, but the result was completely wrong. After some debugging I realized the image was horizontally mirrored relative to what the calibration assumed, because the screenshot workflow had introduced a flip. This led me to revisit why the checkerboard uses an asymmetric grid — 10 columns and 7 rows, one even and one odd — so that the pattern cannot look identical after a 180-degree rotation, which would make it impossible to determine the correct orientation. Understanding that design choice made the whole coordinate system feel much less arbitrary.

Acknowledgement

This project was completed collectively by Junyao Han and Junrui Ding. We consulted the OpenCV documentation for functions including `findChessboardCorners`, `cornerSubPix`, `drawChessboardCorners`, `calibrateCamera`, and `solvePnP`. We also used AI tools for debugging help, implementation guidance, and report drafting support.